Course Discussions

Solution: Valid Anagram

Problem Statement

Given two strings `s` and `t`, return `true` if `t` is an anagram of `s`, and `false` otherwise.

An **Anagram** is a word or phrase formed by rearranging the letters of a different word or phrase, typically using all the original letters exactly once.

Example 1:

```
Input: s = "listen", t = "silent"
Output: true
```

Example 2:

```
Input: s = "rat", t = "car"
Output: false
```

Example 3:

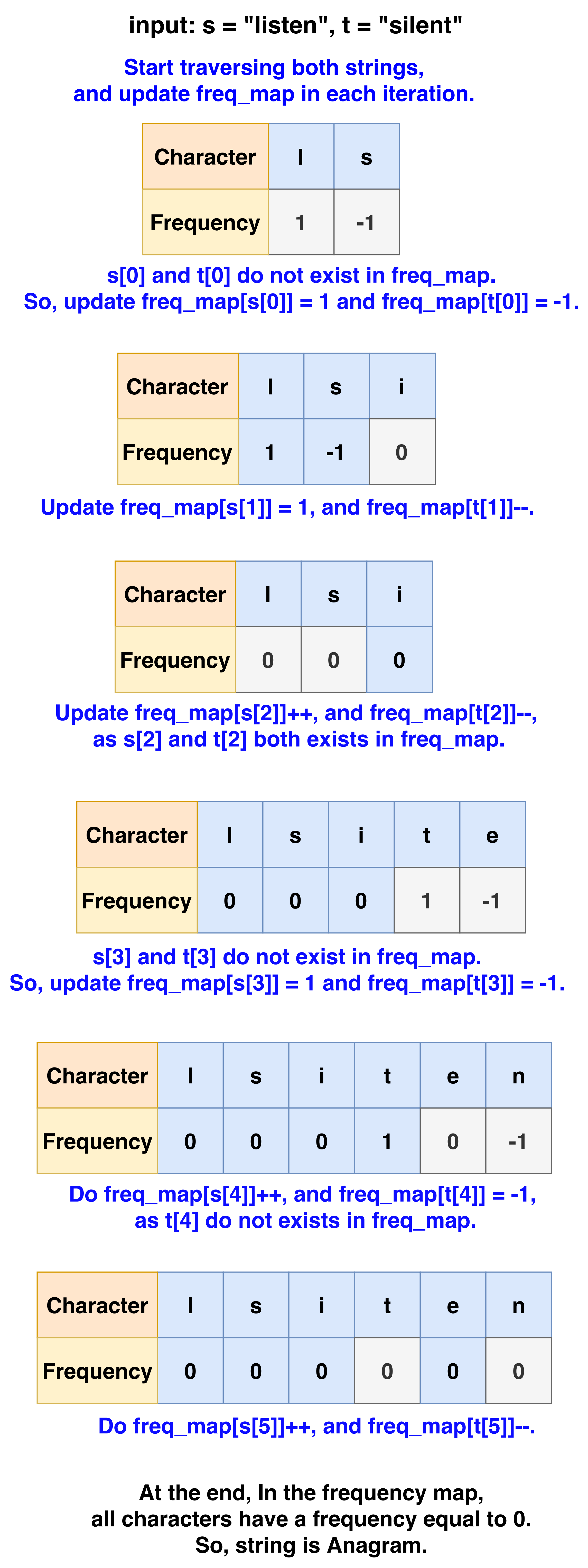
```
Input: s = "hello", t = "world"
Output: false
```

Solution

We can solve this problem by calculating how many times each character appears in both the strings. If the frequency of each character is the same in both the given strings, we can conclude that the strings are anagrams of each other.

We can use a hash map to store the frequency of each character in both strings.

For each character in the string, the frequency of that character in string `s` is incremented and the frequency of that character in string `t` is decremented. After iterating over all characters in the strings, we will check if the frequency of all characters is 0. If it is, the strings are anagrams of each other and the function returns true. Otherwise, it returns false.



input: s = "listen", t = "silent"

Start traversing both strings, and update freq_map in each iteration.

Character	l	s
Frequency	1	-1

s[0] and t[0] do not exist in freq_map. So, update freq_map[s[0]] = 1 and freq_map[t[0]] = -1.

Character	l	s	i
Frequency	1	-1	0

Update freq_map[s[1]] = 1, and freq_map[t[1]]--.

Character	l	s	i
Frequency	0	0	0

Update freq_map[s[2]]++, and freq_map[t[2]]--, as s[2] and t[2] both exists in freq_map.

Character	l	s	i	t	e
Frequency	0	0	0	1	-1

s[3] and t[3] do not exist in freq_map. So, update freq_map[s[3]] = 1 and freq_map[t[3]] = -1.

Character	l	s	i	t	e	n
Frequency	0	0	0	1	0	-1

Do freq_map[s[4]]++, and freq_map[t[4]] = -1, as t[4] do not exists in freq_map.

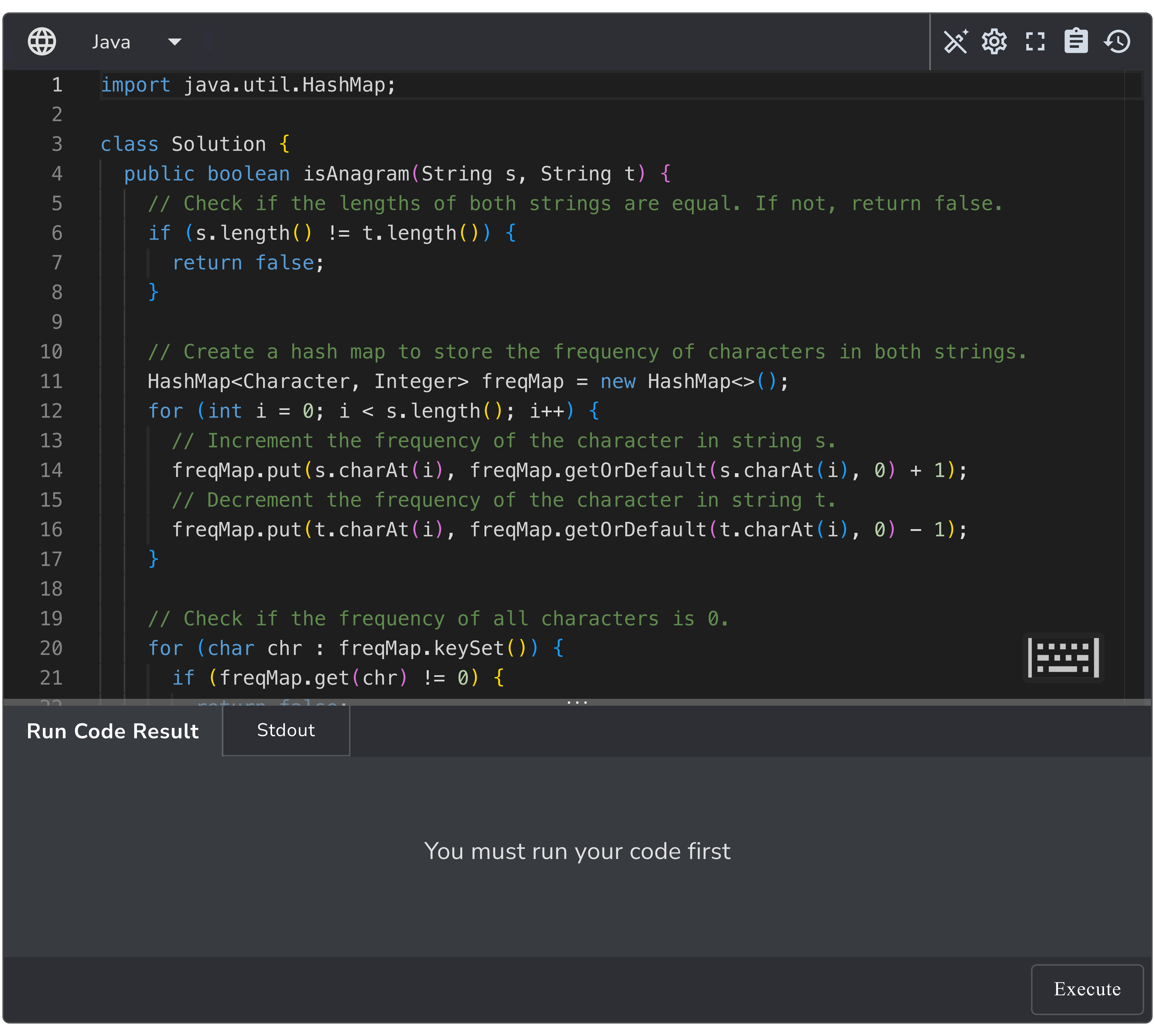
Character	l	s	i	t	e	n
Frequency	0	0	0	0	0	0

Do freq_map[s[5]]++, and freq_map[t[5]]--.

At the end, In the frequency map, all characters have a frequency equal to 0. So, string is Anagram.

Code

Here is the code for this algorithm:



```java
import java.util.HashMap;

class Solution {
  public boolean isAnagram(String s, String t) {
    // Check if the lengths of both strings are equal. If not, return false.
    if (s.length() != t.length()) {
      return false;
    }

    // Create a hash map to store the frequency of characters in both strings.
    HashMap<Character, Integer> freqMap = new HashMap<>();
    for (int i = 0; i < s.length(); i++) {
      // Increment the frequency of the character in string s.
      freqMap.put(s.charAt(i), freqMap.getOrDefault(s.charAt(i), 0) + 1);
      // Decrement the frequency of the character in string t.
      freqMap.put(t.charAt(i), freqMap.getOrDefault(t.charAt(i), 0) - 1);
    }

    // Check if the frequency of all characters is 0.
    for (char chr : freqMap.keySet()) {
      if (freqMap.get(chr) != 0) {
```

Time Complexity

The time complexity of this algorithm is $O(n)$, where n is the length of the strings. This is because the algorithm iterates over each character in the strings once and performs a constant amount of work for each character.

Space Complexity

The space complexity of this algorithm is $O(1)$, as the size of the hash map is proportional to the number of unique characters in the strings. In the worst case, all characters in the strings are unique, so the size of the hash map would be 26 (the number of letters in the alphabet).

However, in most cases, the number of unique characters is much smaller than 26, so the space complexity is closer to $O(k)$, where k is the number of unique characters in the strings.

← Previous
Valid Anagram (Easy)

Next →
Shortest Word Distance (Easy)

☐ Mark as Completed